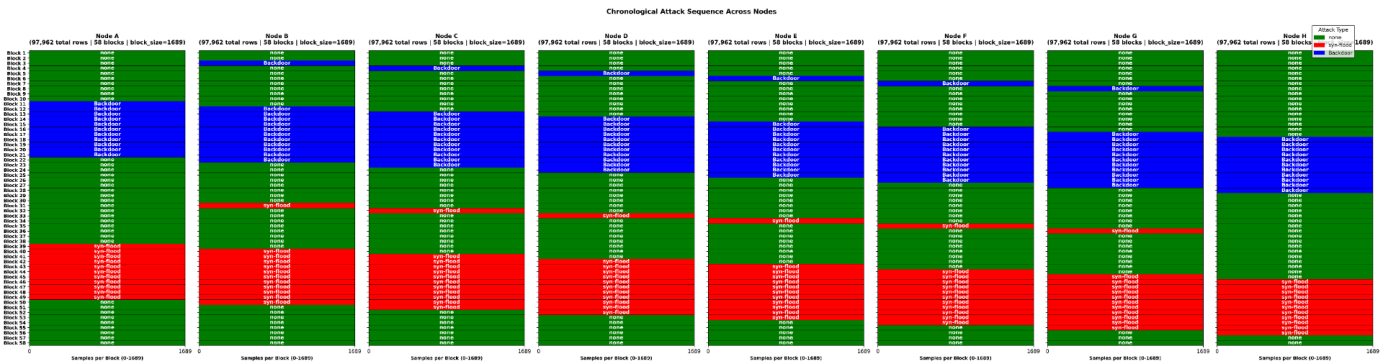


# Kontext vs korrelation

Ein model was komplett auf alle möglichkeiten trainiert ist.

Ergebnisse erklären:

- **Baseline:** Nur lokale Hardwaredaten.
- **Modell 1 (Context):** Eigene HW + aggregierte Nachbarschaftsdaten (HW & Angriffswahrscheinlichkeiten via Sigmoid oder Utility).
- **Modell 2 (Full Context):** Eigene HW + vorberechnete eigene Angriffswahrscheinlichkeit + Nachbarschaftskontext.



58 blöcke je ~1700 zeilen 97000 zeilen je Knoten - Gemischt Originaler datensatz | DAE-Model | TabGAN model - genaue aufteilung.

Train - 70 % zufällig von jedem block wurde fürs training des RF models genutzt - Die parameter sind gleich aber jede Node hat ein eigenes trainiertes Model erhalten

Test - 30 % von jedem block wurde für Test zwecke genutzt (für gesamt ergebnisse wurden einmal komplett über den datensatz getestet für cases 1-Attack 7-None usw. siehe folgende Tabelle:

| Attack Nodes | None Nodes | Test-Blocks                    |
|--------------|------------|--------------------------------|
| 8            | 0          | 18–21, 46–49                   |
| 7            | 1          | 17, 22, 45, 50                 |
| 6            | 2          | 16, 23, 44, 51                 |
| 5            | 3          | 15, 24, 43, 52                 |
| 4            | 4          | 14, 25, 42, 53                 |
| 3            | 5          | 13, 26, 41, 54                 |
| 2            | 6          | 12, 27, 40, 55                 |
| 1            | 7          | 3–8, 11, 28, 31–36, 39, 56     |
| 0            | 8          | 1–2, 9–10, 29–30, 37–38, 57–58 |

mind immer 4 blöcke = mind. >2000 zeilen für testen von z. B. 7 Attacks Nodes und 1 Benign Node

Baseline RF classifier ohne Nachbar Kontext:

Baseline

|                                                 |          |
|-------------------------------------------------|----------|
| === Combined Test Set Performance (df_test) === |          |
| Macro F1-Score:                                 | 0.878141 |
| Mean Squared Error:                             | 0.114226 |
| Log Loss:                                       | 0.237548 |
| ROC-AUC Score:                                  | 0.973026 |

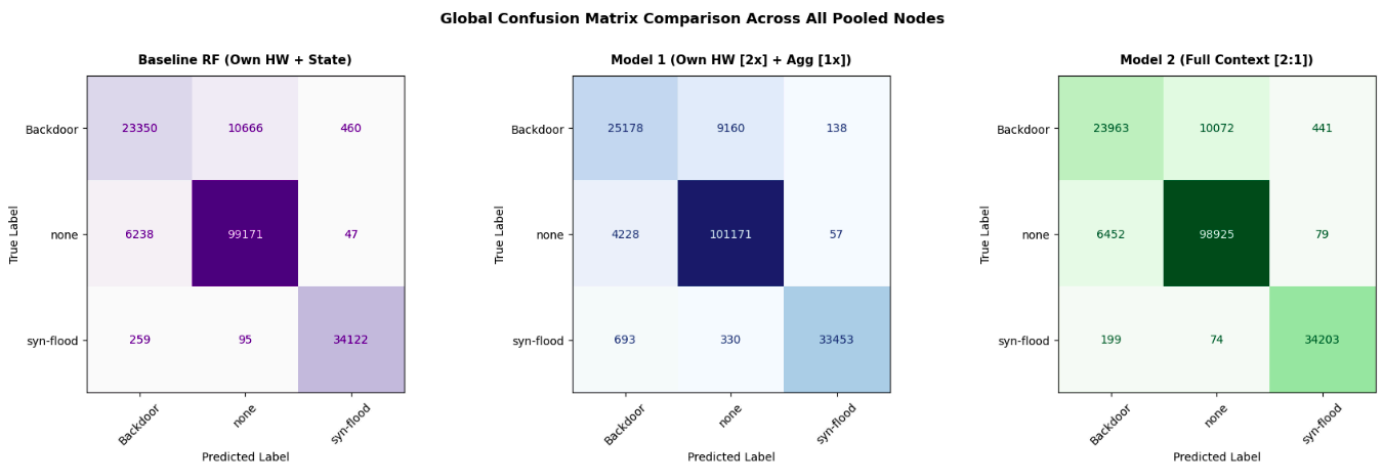
Model 1:

|                                                        |          |
|--------------------------------------------------------|----------|
| === Global Benchmarking Metrics (All Nodes Pooled) === |          |
| Accuracy:                                              | 0.923345 |
| Macro F1:                                              | 0.911000 |
| Weighted F1:                                           | 0.922845 |
| MSE:                                                   | 0.093795 |
| Log Loss:                                              | 0.186730 |
| ROC-AUC (Macro OVR):                                   | 0.983984 |

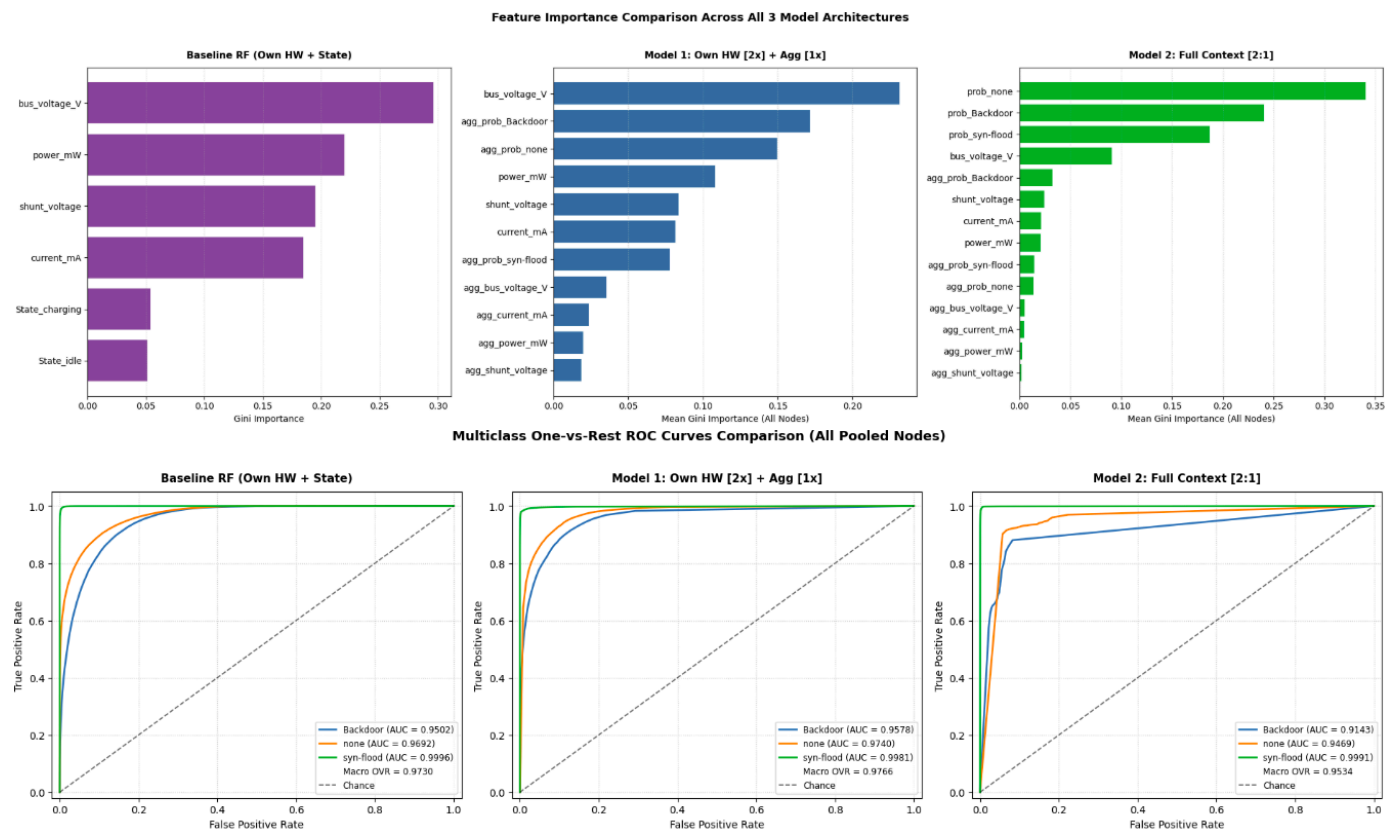
Model 2:

|                                                        |          |
|--------------------------------------------------------|----------|
| === Global Benchmarking Metrics (All Nodes Pooled) === |          |
| Accuracy:                                              | 0.901062 |
| Macro F1:                                              | 0.884247 |
| Weighted F1:                                           | 0.899017 |
| MSE:                                                   | 0.111933 |
| ROC-AUC (Macro OVR):                                   | 0.940173 |

Syn-flood kann er am besten klassifizieren, Benigne probleme und Backdoor - Node E hat die beste mischung aus viel Syn-flood und Mehr None als Backdoor => bestes gesamt Klassifikator erg.



Overall ranking:

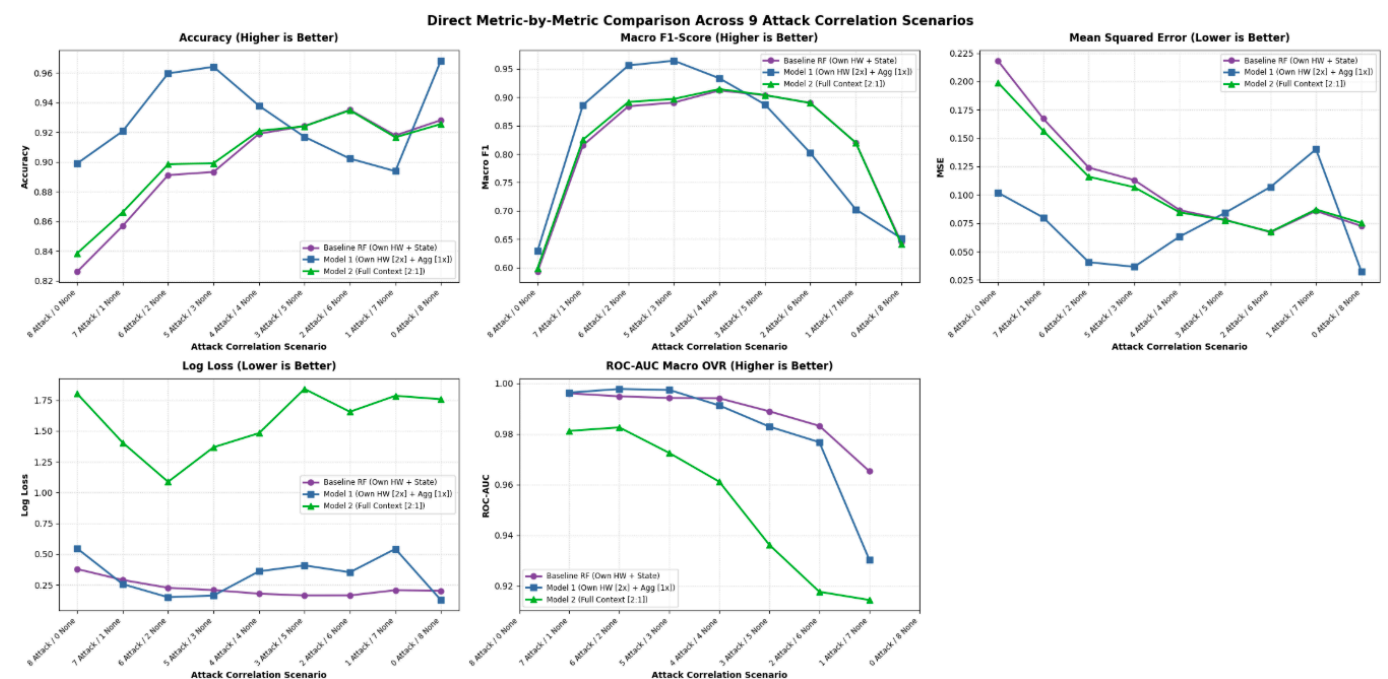


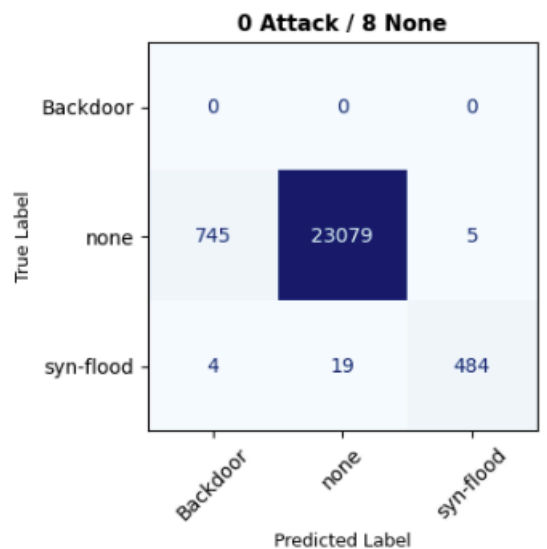
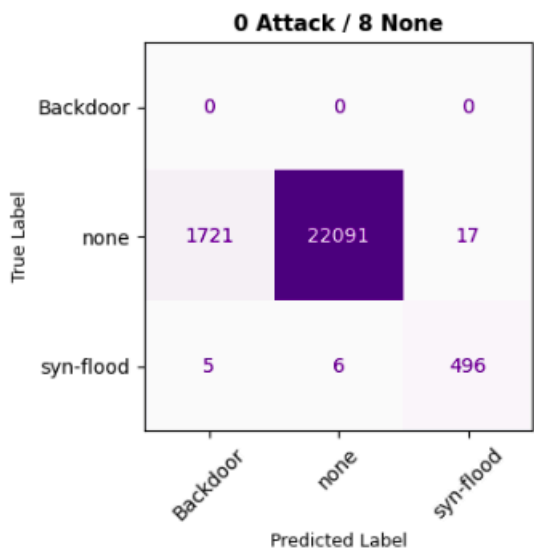
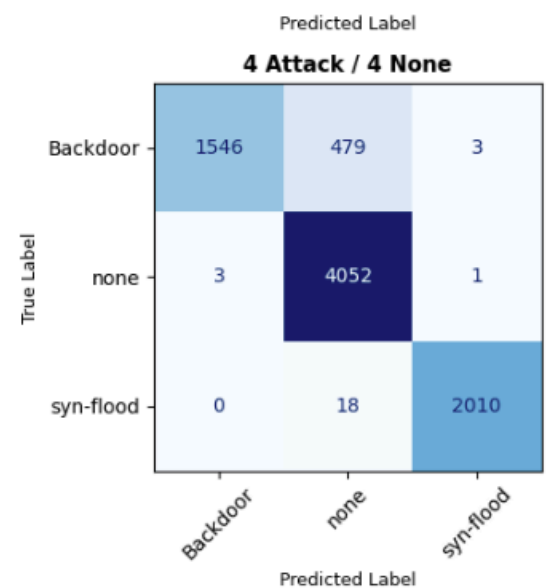
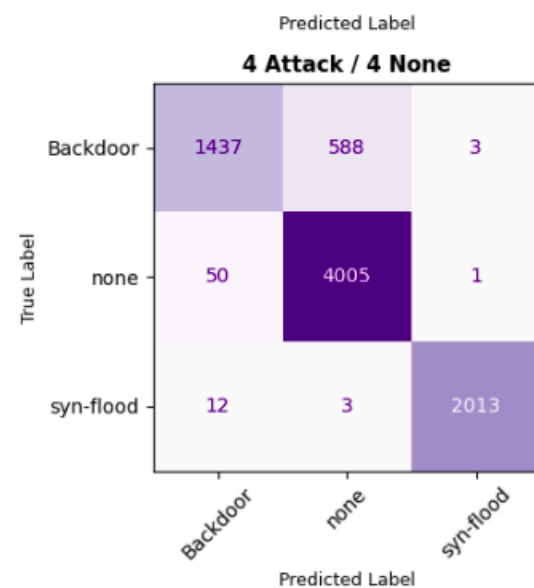
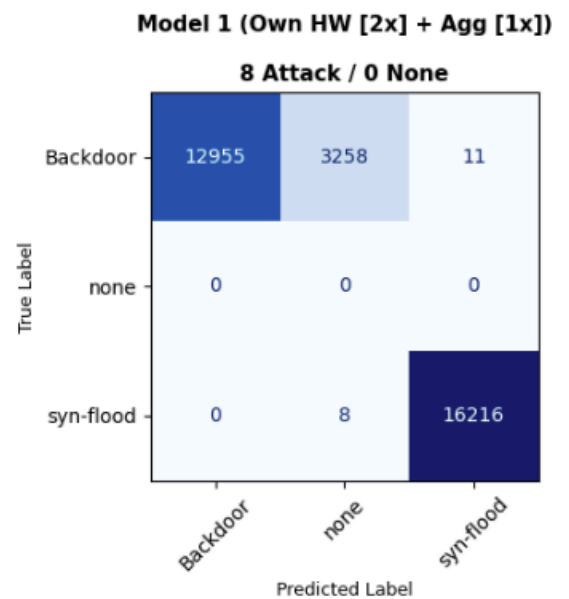
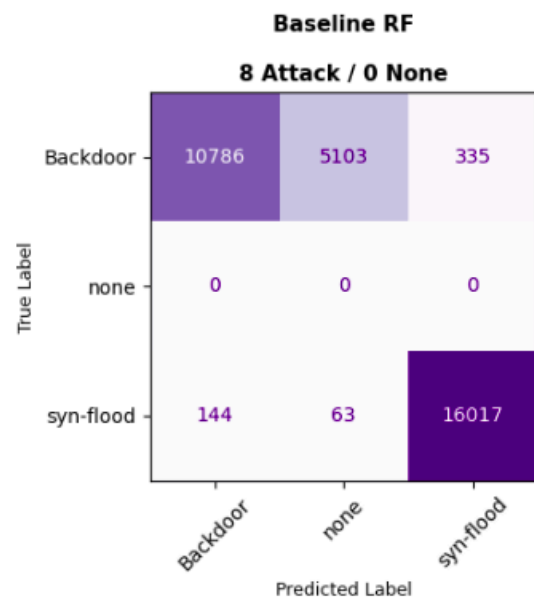
Daraus können wir schließen dann die kontextinformation overall nur minimale verbesserungen bringen:

Nun die analyse ab wann kontextinformationen an sich bessere ergebnisse liefern

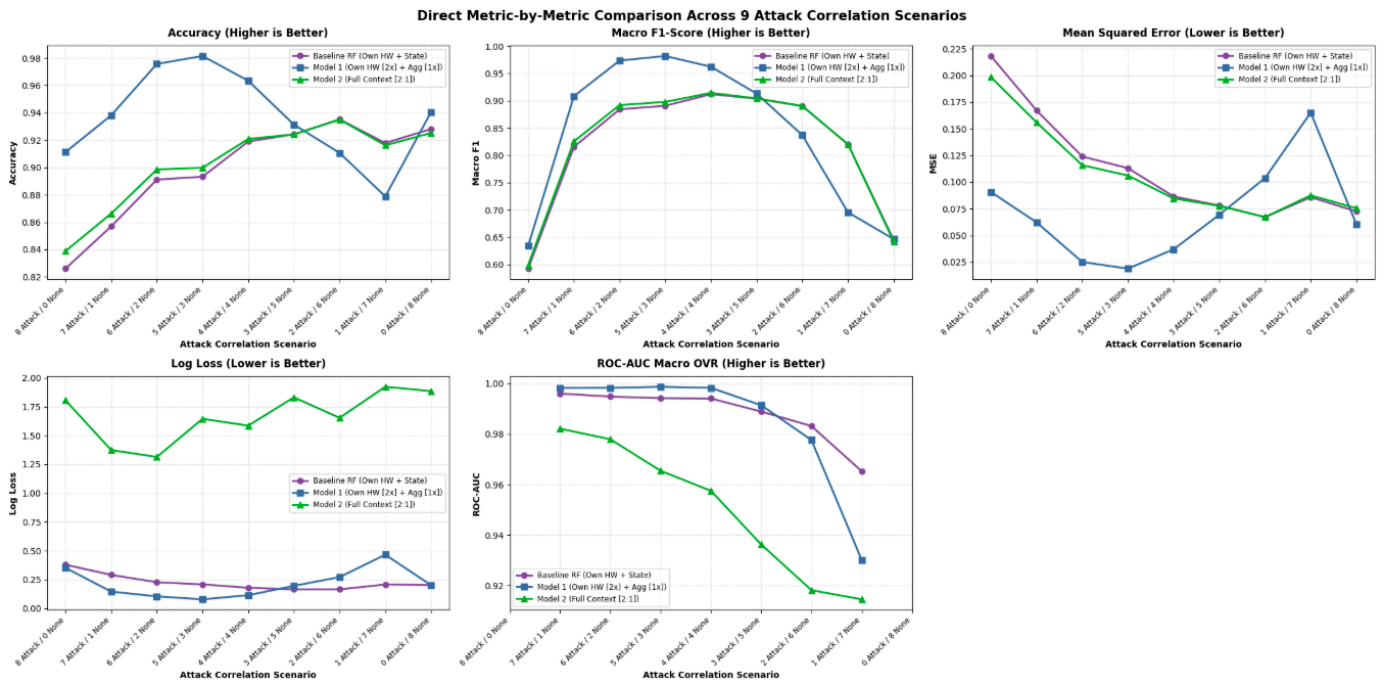
**Ab welchen punkt verbesserte Ergebnisse durch kontext:**

Ergbnis mit max weight x3





Ergbnis mit max weight x10



Muss noch getestet werden

X = baseline : No aggregation. Local ML detection only.

Reference for all comparisons.

O = no\_weights : Aggregated Threat-Reports from all neighbours.

All nodes weighted equally.

O = fixed\_weights : Aggregated Threat-Reports from all neighbours.

Manually defined static weights per node.

O = custom\_weights : Aggregated Threat-Reports from all neighbours.

Dynamic weights via Utility-Theory or Sigmoid function

based on number of positive detections per node.